pulling.

Good luck with the grade, Luis.

> What we have learned so far that we can synthesize into a framework. Using the first experience plus the "pedagogy perspective" work we did. Also leverage Claude or Anthropic's spec-driven development framework and Software development life cycle, which has emerged as innovative and delivers high-quality outcomes, with planning and user stories (or overall planning) imperative before any code is written. List succintly what is the approach we are developing.

Building a framework from prior lessons and pedagogy work.

Working name: Spec-First, Evidence-Driven Development. Requirements, plans and tests come before code. Every claim is proven by evidence. Each step is designed to build the developer's understanding as well as the software.

I didn't find a separate "pedagogy perspective" document in this conversation or your saved notes. I've drawn the learning layer from what we practiced here; if that work lives somewhere else, share it and I'll fold it in.

Sources it combines

- **Spec-driven development:** GitHub's Spec Kit flow of constitution → specify → plan → tasks → implement.
- **Claude Code practices:** explore → plan → code → commit, a `CLAUDE.md` project context file, and test-first work.
- **Classic SDLC discipline:** requirements, design, build, verify, release, and traceability between them.
- **What our project showed:** measure before deciding, check that tests can fail, and triage every review.

Principles

1. **Understand before code.** Planning and user stories are required before any code is written.
2. **Rubric becomes rules.** Success criteria become a constitution that every later step is checked against.
3. **Every requirement traces to a test.** Spec → acceptance criterion → task → test → commit, with nothing orphaned.
4. **Evidence over assertion.** Results come from files and scripts, never typed by hand, and each measurement is itself checked.

5. **Tests must prove they can fail.** Write the test first, watch it fail, then deliberately break the code to confirm the test catches it.
6. **Small, reversible steps.** One task per commit, each with a clear "done when."
7. **Documents lead code.** If reality differs from the spec, update the spec first, then the code.

Lifecycle, with a gate at each step

#	Phase	Output	Gate
0	Constitution	Non-negotiable rules derived from the rubric or standards	Human approves
1	Specify	User stories, requirements, acceptance criteria, reference answers, open questions	Independent review, then human decides the open questions
2	Spike	Small throwaway experiments that test key assumptions	Assumptions confirmed or the spec is revised
3	Plan	Design with the reason for every choice	Independent review, then human approves
4	Tasks	Ordered, test-first checklist with owners	Human approves
5	Build	Failing test → code → refactor → commit, one task at a time	Tests pass, lint clean, mutation-checked
6	Verify	Real measurements on the target machine, a fresh clone, every acceptance criterion checked	Everything ticked
7	Deliver and reflect	README, decision log, lessons learned	Submitted, and lessons fed into the next constitution

Roles

- **Human (owner):** decides, approves each gate, runs real-world measurements, and owns the final judgment.
- **Builder AI (Claude Code):** the only tool that changes the repo; drafts, implements and explains.
- **Reviewer AI (Gemini or another model):** reviews independently and never edits.
- **Every review finding gets a decision:** accept, adjust or reject, with the reason recorded.

Learning layer (pedagogy)

- **Explain "why" at every choice,** so the owner can defend it.
- **The owner states intent first,** even roughly; the AI refines it.

- **Name each lesson as it happens,** such as the NaN trap or a 10 ms timer that can't see Go's speed.
- **Keep reasoning in writing:** the decision log and review records are a learning record as well as an audit trail.
- **Scaffold, then fade:** the AI does more early on, and the owner takes over tasks as confidence grows.

Standard artifacts

`constitution.md` · `spec.md` · `plan.md` · `tasks.md` · `reviews/` · `decisions.md` · `CLAUDE.md` · `results/` · `README.md`

I can turn this into a reusable template repo or a one-page document for your MSDS cohort or GIDS.

Create a spreadsheet from your last answer.

Used artifacts ▸

Your account doesn't offer a Sheets artifact type, so I'll build an Excel file. First, the spreadsheet skill's instructions:

Shared a file, read 2 files, ran 3 commands ▸



I've turned the framework into an Excel workbook, `SFEDD-Framework.xlsx`, with one tab per section of my last answer: Overview, Principles, Lifecycle, Roles, Learning Layer and Artifacts.

Your account doesn't have a spreadsheet type for shareable pages, so this is a regular Excel file. It opens in Excel, Numbers or Google Sheets.

The content is the same as my last answer. Two things are new:

- **Artifacts tab:** each standard file now has a "Created in phase" column showing where it first appears in the lifecycle.
- **Layout:** each sheet has a frozen, filterable header row and prints on one landscape page.

If you'd like to use it as a working checklist for future assignments, I can add a "Status" column to the Lifecycle tab.



We will follow the same steps and build this together, as we did in our last work. Let's follow the framework we developed and build the deliverables together, step by step, as before. Let's use the assignment to produce and start practicing with the model you recommended. The objective is to complete the assignment using spec-driven development. We must use the Programming Assignment Rubric as part of the requirements. Task: Let's take a step-by-step approach so I can also meet the learning objectives for developing robust code.

Created 4 files, shared 4 files, and 29 more steps ▸

Phases 0 and 1 are drafted (constitution and spec), and the spike behind them succeeded.

The due date is tomorrow (10/25). Last week's full process took about 11 hours. To finish in time, I suggest we keep every step of the framework but run it fast:

- **One combined Gemini review** for the spec and plan, instead of two separate rounds.
- **Core scope first:** PyTorch network, parity check, Python vs. Go timings, executables, and README.
- **Stretch goals only if time remains:** the convolutional network and the ONNX Runtime comparison.

If your course allows late work or extensions, tell me and we can slow down.

What the spike proved, before any planning

A Python neural network exported to ONNX and run in Go with GoMLX gave **identical predicted labels**, with probabilities within **0.00000036** of Python's. That's the assignment's core claim, shown to work. It also found three problems we'd otherwise have hit in the middle of building:

1. **GoMLX now requires Go 1.27.** Your Mac has 1.27.2, so you're fine.
2. **GoMLX's speed-up for dense layers ("fusion") breaks on exported models,** so it has to be switched off.
3. **GoMLX can't run classic-ML operations,** so Go reads the probabilities and picks the winning class itself.

One issue only you can decide. The assignment says to use **free versions** of AI tools. Check whether your Claude plan counts under that wording. If it doesn't, consider saying so openly in the README or asking your instructor. That's question Q7 in the spec, and the repository also needs your AI conversations saved as PDF or text.

Your three actions

1. **Create the GitHub repository** (Public, empty, no README), for example `go-onnx-production`. Then install the Claude app on it the same way as last time: github.com/apps/claude/installations/select_target → **mioluigi** → Only select repositories → add the new repo. Send me the repository name and I'll push these documents.
2. **Read the spec** (`spec.md`, about 10 minutes), then upload all four files to gemini.google.com with:

   > You are a strict grader and senior Go/ML engineer. Review spec.md against constitution.md and assignment-and-rubric.md, using the spike results in 001-gomlx-onnx.md. What is ambiguous, missing, untestable, or risky? Which acceptance criteria would a strict grader say are not met? Cite section numbers.
3. **Answer Q1–Q8** in spec section 11. "Accept all" works, or tell me which to change.

While you do that, I'll check the riskiest new assumption: whether a **PyTorch** network exports to ONNX in a form GoMLX can run. The spike used scikit-learn.

Sources:

- onnx-gomlx on pkg.go.dev
- GoMLX on pkg.go.dev

Download all

ok, back on step 1. I am creating in github the go-onnx-production respository, walk me through it and the rest of the step that i need to configure as last time.

Loaded tools

Here's the setup, in order. It's simpler than last time because the Claude app is already installed on your account; you only need to add the new repository to it.

Step 1 — Create the empty repository

1. Go to **github.com/new**
2. **Repository name:** `go-onnx-production`
3. **Public**, so the grader can clone it.
4. Leave **all** boxes unchecked (no README, no .gitignore, no license). It must be empty, because my workspace already has the first commit.
5. Click **Create repository**.

Step 2 — Give Claude access to it

1. Go to **github.com/settings/installations**
2. Next to **Claude**, click **Configure**.
3. Under **Repository access**, keep **Only select repositories**, open **Select repositories**, and add `go-onnx-production`. Both `go-anscombe-stats` and `go-onnx-production` should now be listed.
4. Click **Save**. Unlike last time, Save will be active here because you changed something.

Step 3 — Tell me "done"

I'll push the first commit (constitution, spec, spike record, assignment summary).

Step 4 — Clone it on your Mac (after I push)

This time you clone instead of unzipping:

```bash
cd ~/Projects
git clone https://github.com/mioluigi/go-onnx-production.git
cd go-onnx-production
code .
```

While you do steps 1–3, I'll check the open item from last night: whether a **PyTorch** network exports to ONNX in a form GoMLX can run.

Used Add repo, ran 8 commands ▸

done

Register repo root ▸

The setup is done and the first push worked: GitHub has all four commits on `main`.

Commit	Contents
1	Constitution, assignment summary, spike record, draft spec
2	Spike addendum: PyTorch-style models work in GoMLX
3	Spec updated with those findings

Clone it on your Mac. This replaces the unzipping from last time:

```bash
cd ~/Projects
git clone https://github.com/mioluigi/go-onnx-production.git
cd go-onnx-production
git log --oneline
code .
```

You should see the three commits, and VS Code will open the project.

While you were setting up, I closed the open PyTorch question. PyTorch's two export styles both run correctly in GoMLX, matching the reference within 0.00000006. GoMLX's dense-layer speed-up ("fusion") also works with them; only scikit-learn's exports needed it switched off. That's good support for choosing PyTorch (Q1). The spike record and spec now include this. One part still has to happen on your Mac: exporting the real trained model from PyTorch.

Next, the same two actions as last week:

1. **Gemini review of the spec.** At gemini.google.com, upload `spec.md`, `constitution.md`, `001-gomlx-onnx.md`, and `assignment-and-rubric.md`, then send:

   > You are a strict grader and senior Go/ML engineer. Review spec.md against constitution.md and assignment-and-rubric.md, using the spike results in 001-gomlx-onnx.md. What is ambiguous, missing, untestable, or risky? Which acceptance criteria would a strict grader say are not met? Cite section numbers.

2. **Your decisions on Q1–Q8** (spec section 11). Reply "accept all" or name your changes. The one I can't decide for you is Q7, whether your AI tools meet the assignment's "free versions" wording.

The assignment is due today, so you could send your Q answers now and paste Gemini's review when it's ready. I'll start the plan as soon as I have your answers.

```
Cloning into 'go-onnx-production'...
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 20 (delta 4), reused 20 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (20/20), 13.18 KiB | 4.39 MiB/s, done.
Resolving deltas: 100% (4/4), done.
16a54fd (HEAD -> main, origin/main, origin/HEAD) docs: update spec 001 findings
with the PyTorch spike results
d3a582f docs: spike 001 addendum: PyTorch-style ONNX graphs run in GoMLX
7e98d93 docs: add constitution, assignment summary, spike 001, and draft spec 001
```

The clone matches GitHub exactly, with the same three commits, so your Mac, my workspace, and GitHub are in sync.

To save time today, you can start a slow install on your Mac while Gemini reviews the spec. **PyTorch takes a few minutes to download**, and starting it now also tells me which exact versions to pin:

bash

```
cd ~/Projects/go-onnx-production
python3 -m venv .venv
source .venv/bin/activate
pip install torch torchvision onnx onnxruntime onnxscript
```

When it finishes, paste the output of:

bash

```
python -c "import torch, torchvision, onnx, onnxruntime; print(torch.__version__,
```

`.venv` is your local Python environment and shouldn't be committed. I'll add it to `.gitignore` in the first build task.

Meanwhile, in parallel:

1. **Gemini review:** upload the four files (`spec.md`, `constitution.md`, `001-gomlx-onnx.md`, `assignment-and-rubric.md`) with the prompt from my last message.
2. **Your answers to Q1–Q8:** "accept all" or your changes, plus your call on Q7 (the "free versions" wording).

Send whichever arrives first. With your answers I can start the plan, and Gemini's points can be folded in when they arrive.



Gemini response

Edited a file, created a file, and 6 more steps ▸

I've updated the spec with Gemini's review and pushed it. Run `git pull` to get spec v2 and the review record, `docs/reviews/001-spec-review.md`.

Decision	Gemini's finding	What changed in the spec
✅ Accept	Images need a different input pipeline from the spike's table data	New **FR-3a**: one written input rule (784 values, `float32`, pixel ÷ 255) applied the same way in Python and Go, and tested
🔶 Adjust	A Windows `.exe` can't include XLA	**Tested:** Windows builds only with the pure-Go backend, because XLA needs C code. Spec updated. The "apples to oranges" benchmark worry doesn't apply, since all timing happens on your Mac
✅ Accept	"Start-up cost" was vague, and GoMLX compiles on first use	**FR-7** now times three stages: loading the model, the first prediction (including compilation), and later predictions
✅ Accept	NaN outputs weren't rejected	Any NaN or infinite probability fails the parity check
✅ Accept	Monte Carlo should use `go test -bench`	Required in **FR-8**
✅ Accept	README tables need a script	New **NFR-8**, the same approach as last week
🔶 Adjust	Profile files unspecified	The tools and folder are now named; exact file names belong in the plan
🔶 Adjust	`.app` layout missing	Cross-compiling is clarified. The `.app` layout is a "how" for the plan; we reuse last week's tested script
❌ Reject	Whole-program memory method undefined	The spec already said "the Week 4 approach". I made the wording explicit

The lesson this time: Gemini's Windows point was **half right**. The build limit was real, but the benchmarking worry came from a misreading. A two-minute test separated the two.

Two things still needed from you before the plan:

1. **Your answers to Q1–Q8** (spec section 11). "Accept all" works, or name your changes. Q7 (the "free versions" wording) is yours alone to decide.
2. **The PyTorch install versions,** if it's finished:

```bash
python -c "import torch, torchvision, onnx, onnxruntime; print(torch.__version_
```

accept all

Working out exec reuse, timing, and the task breakdown ▸

"Accept all" covers Q1–Q6 and Q8. For Q7, I'll record the dialog-export plan. Whether your AI tools count as "free versions" stays a statement only you can make, so the README will have a short line for you to fill in. Marking the spec approved:

Created 2 files, ran 6 commands ▸

The spec is approved and the plan and task list are pushed. Run `git pull`, then read `plan.md` and `tasks.md` in `specs/001-onnx-go-inference/`.

Plan highlights to understand before approving

- **§3, data:** the test set uses MNIST's **official files, unchanged**, in their original IDX format, and PyTorch's predictions are saved in the same format. One small Go reader handles all of it, and no conversion step can introduce errors.
- **§4, compile once, predict many times:** GoMLX compiles the model once and reuses it for every request. The README example's approach recompiles on each call, which is the opposite of what a production service needs.
- **§6, the program `predict-digits`:** with no arguments it checks parity on all 10,000 images and prints PASS/FAIL with timings. With `-input file.csv` it predicts your own digits.
- **§8, fair timing:** four measurements defined identically for PyTorch, ONNX Runtime, GoMLX-XLA and GoMLX-pure-Go: model load, first prediction, 10,000 images in one batch, and one image at a time. One image at a time is what each of your "thousands of users" actually experiences.

The riskiest step comes first (T03). Before building on it, we run the real PyTorch export on your Mac and check it in Go. If PyTorch exports something the spike didn't cover, we find out in the first hour, not the last.

Deadline trade-off. Normally Gemini reviews the plan before any code. To save time, I propose Gemini reviews it **while** I do the low-risk setup tasks (T01, T02, T04), and anything it finds still gets triaged. I'll record this in the decision log.

Your next actions:

1. Reply **"plan approved"**, or ask "why X?" about anything.
2. **T00:** paste the PyTorch version line:

```bash
cd ~/Projects/go-onnx-production && source .venv/bin/activate
python -c "import torch, torchvision, onnx, onnxruntime, onnxscript; print(torc
```

If the install never finished or `onnxscript` is missing, run `pip install torch torchvision onnx onnxruntime onnxscript` first.
3. **Optional, in parallel:** send `plan.md`, `tasks.md` and `spec.md` to Gemini with:

> Review plan.md and tasks.md: do they fully implement spec.md? What is risky, missing, or not idiomatic Go? Cite sections.

plan approved

Settling on the ONNX export settings and IR version ▸
Plan approved. Starting **T01 (scaffolding)**.

One constraint from my workspace: it can't reach Google's Go module servers, so I build against GitHub mirrors in a separate, git-ignored module file. The committed `go.mod` stays clean, and you'll run `go mod tidy` once on your Mac to produce the real checksum file. I'm recording that, and the parallel-review decision, in the decision log.

Created 4 files, ran 2 commands ▸

Traceback (most recent call last):
File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'torch'
(base) levarela@MacBookPro go-onnx-production % cd ~/Projects/go-onnx-production && source .venv/bin/activate

```
  python -c "import torch, torchvision, onnx, onnxruntime, onnxscript;
print(torch.__version__, torchvision.__version__, onnx.__version__,
onnxruntime.__version__, onnxscript.__version__)"
source: no such file or directory: .venv/bin/activate
Traceback (most recent call last):
  File "<string>", line 1, in <module>
```

The virtual environment was never created; the earlier install step didn't run. Two small things show in your prompt:

- `(base)` means **Anaconda's base environment** is active. It's best to step out of it so this project uses its own clean Python.
- `.venv` doesn't exist yet in `go-onnx-production`.

Run these **one block at a time.** The `pip install` downloads PyTorch, which takes a few minutes:

bash

```
conda deactivate
cd ~/Projects/go-onnx-production
python3 --version
python3 -m venv .venv
source .venv/bin/activate
```

Your prompt should now start with `(.venv)`. Then:

bash

```
pip install --upgrade pip
pip install torch torchvision onnx onnxruntime onnxscript
```

When it finishes:

bash

```
python -c "import torch, torchvision, onnx, onnxruntime, onnxscript; print(torch._
```

Paste the `python3 --version` line and the final version line. If `pip` reports an error mentioning your Python version, paste that too. I'm writing the training program (T02) meanwhile.

Created 3 files, ran 8 commands ▸

The extra-bytes mutation was caught, but the size-guard mutation wasn't, which shouldn't be possible. A gap like this could be in the test, or in my experiment. I'll find out which:

Created 2 files, ran 5 commands ▸

Done so far: T01, T02, T04, T05 and T06, each pushed as its own commit. The `idx` and `parity` packages are finished and tested (96.7% and 100% coverage), and both mutation checks were caught.

Next is T03, the go/no-go gate, and it runs on your Mac. It trains the real PyTorch model, exports it, and checks it. Once your `pip install` has finished:

```bash
cd ~/Projects/go-onnx-production
git pull
source .venv/bin/activate
python python/train_export.py
```

It takes about 1–3 minutes. The first run downloads MNIST (about 12 MB). Watch for these lines:

- `test accuracy:` should be about **0.97–0.98**
- `ONNX Runtime vs PyTorch: largest probability difference` should be **well under 1e-05**
- `saved:` followed by five files in `assets/`

Then save the exact package versions and push everything:

```bash
pip freeze | grep -iE "^(torch|torchvision|onnx|onnxruntime|onnxscript|numpy)==" >
cat python/requirements.txt
git add assets python/requirements.txt
git commit -m "T03: trained model, ONNX export, MNIST test set, PyTorch prediction
git push
```

Paste the full output of `train_export.py` here, whether it succeeds or fails. As soon as your push lands, I'll run the Go parity check on all 10,000 images in my workspace. That's the gate the rest of the plan depends on.

If the export step fails, the error message is exactly what we need. The PyTorch exporter is the one piece I couldn't run here, so this is where a surprise would show up.

2.14.1 0.29.1 1.23.2 1.31.0 0.7.2

Checking library version compatibility for the setup.
Got the versions: **PyTorch 2.14.1, torchvision 0.29.1, onnx 1.23.2, ONNX Runtime 1.31.0, onnxscript 0.7.2.** T00 is done.

Go straight on to **T03** from my previous message: `git pull`, then `python python/train_export.py`, then the `pip freeze` line and the push. Paste the training output when it finishes.

Meanwhile I'm writing the core Go package, `inference`, which T07 and T08 cover.

Created 2 files, edited a file, ran 10 commands, 1 failed ▸

Pasted text.txt

TXT

Read attachment.txt ▸
This is a common macOS Python problem, not a problem with our code. Python can't verify the download site's security certificate because this Python install doesn't have its certificate bundle set up. (It's also **Python 3.14**, not the 3.12 from last week. That's fine; I'll record it.)

Fix it inside the project's environment, then rerun:

bash

```
cd ~/Projects/go-onnx-production
source .venv/bin/activate
pip install certifi
export SSL_CERT_FILE="$(python -m certifi)"
python python/train_export.py
```

The `export` line tells Python to trust the standard certificate bundle that `certifi` provides. It lasts for this Terminal window only. The first download source works with it; the second (yann.lecun.com) is dead, which is expected.

Paste the output when it finishes, then do the `pip freeze` / commit / push steps from before.
Committing T08. The committed `go.mod` gets the three pinned GoMLX requirements, and the checksum file will come from `go mod tidy` on your Mac:

Pushed to a branch ▸

T08 is pushed. While you rerun training, I'll write **T13, the Python timing program.** It doesn't depend on the trained model's contents, and it measures exactly what the Go program will measure:

Created 3 files, read a file, ran 10 commands ▸

Pushed: T09 (Monte Carlo benchmarks) and T13 (Python timing program). I also moved the two PyTorch model classes into `python/mnist_model.py`. Training behaves exactly as before, but timing PyTorch no longer counts torchvision's import.

T03, your turn. Run these in Terminal, one block at a time:

```
cd ~/Projects/go-onnx-production
source .venv/bin/activate
export SSL_CERT_FILE="$(python -m certifi)"
git pull
python python/train_export.py
```

If training finishes, three quick checks follow. The first records the Python package versions. The second runs the new Python timing program once on the real model (about a minute). The third creates `go.sum`.

```
pip freeze | grep -iE "^(torch|torchvision|onnx|onnxruntime|onnxscript|numpy)=="
> python/requirements.txt
python python/bench_python.py --predict-only
go version
go mod tidy
go test ./...
```

`go mod tidy` may download Go 1.27.2 on its own the first time; that's expected, because GoMLX requires it.

Then commit in two steps and push:

```
git add assets python/requirements.txt
git commit -m "T03: trained model, MNIST test set, PyTorch predictions"
git add go.mod go.sum
git commit -m "T01: go.sum from go mod tidy on the Mac"
git push
```

Paste all the output here, even if a step fails. Once your push lands, I'll run the Go parity check on all 10,000 images. Meanwhile I'm writing the program, T10/T11.

Created 5 files, edited 2 files, ran 18 commands ▸

```
Collecting certifi
  Downloading certifi-2026.10.23-py3-none-any.whl.metadata (2.5 kB)
Downloading certifi-2026.10.23-py3-none-any.whl (135 kB)
Installing collected packages: certifi
Successfully installed certifi-2026.10.23
PyTorch 2.14.1
100.0%
100.0%
100.0%
100.0%
epoch 1/5: training loss 0.4142
epoch 2/5: training loss 0.1792
epoch 3/5: training loss 0.1276
epoch 4/5: training loss 0.0978
epoch 5/5: training loss 0.0772
training time: 3.2 s
test accuracy: 0.9711 on 10000 images
[torch.onnx] Obtain model graph for WithProbabilities([...] with
torch.export.export(..., strict=False)...
[torch.onnx] Obtain model graph for WithProbabilities([...] with
torch.export.export(..., strict=False)... ✅
[torch.onnx] Run decompositions...
[torch.onnx] Run decompositions... ✅
[torch.onnx] Translate the graph into ONNX...
[torch.onnx] Translate the graph into ONNX... ✅
[torch.onnx] Optimize the ONNX graph...
[torch.onnx] Optimize the ONNX graph... ✅
ONNX export: {'ir_version': 10, 'opsets': {'ai.onnx': 18}, 'operations': {'Gemm', 'Relu',
'Softmax'}, 'inputs': ['input'], 'outputs': ['probabilities']}
ONNX Runtime vs PyTorch: largest probability difference 1.49e-06
saved: mnist_mlp.onnx (438 KB), mnist_mlp.pt (430 KB), pytorch-probabilities-idx2-
float.gz (374 KB), t10k-images-idx3-ubyte.gz (1610 KB), t10k-labels-idx1-ubyte.gz (4
KB)
```

Validating training results and operator fusion compatibility.

Training worked: 97.11 % test accuracy. The export uses only Gemm, Relu and Softmax, which are the operations the spike proved in GoMLX, and ONNX Runtime matches PyTorch to 1.5e-6.

Training and export worked: **97.11 % test accuracy.** The ONNX graph uses only Gemm, Relu and Softmax, the operations the spike already proved in GoMLX, and ONNX Runtime matches PyTorch to 1.5e-6.

Carry on with the rest of T03, starting at the `pip freeze` block: `pip freeze`, `bench_python.py --predict-only`, `go version`, `go mod tidy`, `go test ./...`, then the

two commits and `git push`. Paste the output. I'm holding my next commits so your push won't be rejected.

Created 2 files, edited a file, ran 21 commands ▸

I've written the program and its measurement and build tools. They're committed but not pushed yet, so your T03 push won't be rejected:

- **T10/T11,** `predict-digits`. With no arguments it checks parity with PyTorch on all 10,000 images, then times loading, the first prediction, all 10,000 images in one call and one image at a time, the same four measures as the Python timing program. `-input` predicts digits from a CSV of pixels and names the line and value of any bad entry. Every test passes. I deliberately broke the code in six ways and a test caught each one.
- **T14.** `scripts/measure.sh` runs every measurement in one command, and `scripts/tables.sh` turns the results into the README tables.
- **T16.** Build scripts for the Mac program, the Windows `.exe` and the `.app`. The `.exe` builds and the native program runs in my workspace. The `.app` can only be built on your Mac.

Things I found along the way, all recorded in the decision log:

- **XLA default and fallback.** The program uses XLA by default on your Mac. XLA downloads a plugin from GitHub the first time it runs, and that download is blocked in my workspace. So if XLA can't start, a no-argument run falls back to pure Go with a warning instead of failing. If you ask for `-backend xla` and it can't start, the program reports an error.
- **Backend names.** GoMLX's pure-Go backend calls itself "Go Backend" rather than "go". A test caught this, and the program now reports the short names `go` and `xla`.
- **An early speed warning.** In my workspace the pure-Go backend is about 36 times slower than PyTorch on the full test set. XLA on your Mac will settle the comparison, and it matters for the recommendation.

Once your T03 push lands, I'll run the parity check on your real model across all 10,000 images, then push my commits. Because XLA support adds a new dependency, you'll need to `git pull` and run `go mod tidy` once more. After that comes the XLA check and the measurement run (T12, T15); I'll send those commands then.

Ran 2 commands ▸

I pushed my four new commits (T10, T11, T14, T16) to a separate branch, `claude/t10-t16`, not to `main`. GitHub may offer a "Compare & pull request" button for that branch. Ignore it; I'll merge it myself.

Pushing to `main` now would cause two problems:

- Your T03 push would be rejected, because `main` would have commits you don't have.
- `go.mod` would conflict: my commits add the XLA dependency to it, and your `go mod tidy` changes it too.

Your T03 steps don't change: finish the `pip freeze` block, then commit and push to `main`. Once it lands, I'll put my commits on top of yours, check parity on all 10,000 images with your real model, and push `main`. After that, you'll pull and run `go mod tidy` once more to pick up the XLA dependency.

Compacted conversation

What T03 do I need run now in Terminal, one block at a time?

Preparing the remaining setup steps for the next session.
You've already trained the model, so four blocks remain. Paste each one, wait for it to finish, then paste the next.

1. Open the project and its Python environment (needed again only if this is a new Terminal window):

```
cd ~/Projects/go-onnx-production
source .venv/bin/activate
```

2. Record the Python versions, then time the model once:

```
pip freeze | grep -iE "^(torch|torchvision|onnx|onnxruntime|onnxscript|numpy)=="
> python/requirements.txt
cat python/requirements.txt
python python/bench_python.py --predict-only
```

The last command should show PASS for both PyTorch and ONNX Runtime.

3. Create `go.sum` and run the Go tests:

```
go version
go mod tidy
go test ./...
```

All three packages should say `ok`.

4. Commit and push:

```
git status --short
git add assets python/requirements.txt
git commit -m "T03: trained model, MNIST test set, PyTorch predictions"
git add go.mod go.sum
git commit -m "T01: go.sum from go mod tidy on the Mac"
git push
```

`git status` should list the five files in `assets/`, `python/requirements.txt`, `go.mod` and `go.sum`, and nothing else.

Paste all the output here, including anything that fails. After your push I'll run the 10,000-image parity check with your model.
The hook compares my local `main` with GitHub's `main`. I'll move my work onto the local branch that matches the one already on GitHub, and reset local `main` to GitHub's. Nothing on GitHub changes, and your push stays clear.

Ran a command ▸
Nothing has changed for you. My unmerged work sits on its own branch, `claude/t10-t16`, which is now fully on GitHub. GitHub's `main` still ends at T09, which you already have, so your T03 push will go through.

Run the four blocks from my last message and paste the output here. When your push lands, I'll put my commits on top of yours, check parity on all 10,000 images with your model, and push `main`.

```
go version go1.27.2 darwin/arm64
ok  github.com/mioluigi/go-onnx-production/idx 0.262s
ok  github.com/mioluigi/go-onnx-production/parity 0.260s
```

Check what GitHub main contains now ▸
Your `go test` output lists `idx` and `parity` but not `inference`. That means your copy is several commits behind GitHub: you don't have `inference` (T07/T08), the benchmarks (T09) or `bench_python.py` (T13). The `git pull` I listed with training didn't run. Because `inference` was missing, `go mod tidy` found no outside packages to record, so `go.sum` probably wasn't created either.

Your trained model and data in `assets/` are fine and don't need retraining. Don't run block 4 yet. Run these instead, one block at a time.

1. Undo `go mod tidy`'s change to `go.mod`, then get the missing commits:

```
cd ~/Projects/go-onnx-production
source .venv/bin/activate
git checkout -- go.mod
git pull
```

2. Time the model once (this should show PASS for PyTorch and ONNX Runtime):

```
python python/bench_python.py --predict-only
```

3. Run `go mod tidy` again, now that the package that uses GoMLX is here, then test:

```
go mod tidy
ls go.sum
go test ./...
```

This time `inference` should appear as `ok`, along with `idx` and `parity`. The first `go test` downloads GoMLX, so it may take a minute or two.

4. Commit and push:

```
git status --short
git add assets python/requirements.txt
git commit -m "T03: trained model, MNIST test set, PyTorch predictions"
git add go.mod go.sum
git commit -m "T01: go.sum from go mod tidy on the Mac"
git push
```

Paste the output of all four blocks, including anything that fails.

```
M go.mod
?? assets/
?? go.sum
?? python/requirements.txt
[main 1a118e7] T03: trained model, MNIST test set, PyTorch predictions
 6 files changed, 6 insertions(+)
 create mode 100644 assets/mnist_mlp.onnx
 create mode 100644 assets/mnist_mlp.pt
 create mode 100644 assets/pytorch-probabilities-idx2-float.gz
 create mode 100644 assets/t10k-images-idx3-ubyte.gz
 create mode 100644 assets/t10k-labels-idx1-ubyte.gz
 create mode 100644 python/requirements.txt
```

```
[main 37c2297] T01: go.sum from go mod tidy on the Mac
 2 files changed, 103 insertions(+)
 create mode 100644 go.sum
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 10 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (14/14), 2.34 MiB | 2.98 MiB/s, done.
Total 14 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 2 local objects.
To https://github.com/mioluigi/go-onnx-production.git
   c0c2bfc..37c2297  main -> main
```

Show more

Resolving module dependencies and cleaning up placeholder assets.

Your push landed: the assets plus a 103-line `go.sum`. I'll swap my stand-in files for your real ones, put my commits on top of yours, and run the parity check on all 10,000 images.

Ran 10 commands ▸

Everything is on GitHub's `main`: your T03 commits, then T10, T11, T14 and T16, then the gate result. The only leftover is the empty-of-new-work branch `claude/t10-t16`, which GitHub won't let me delete from here. You'll delete it in step 1 below.

Next: T12 (XLA on your Mac), then T15 (all measurements). One block at a time; stop and paste if anything fails.

1. Get the new code and remove the leftover branch:

```
cd ~/Projects/go-onnx-production
source .venv/bin/activate
git pull
git push origin --delete claude/t10-t16
```

2. T12: record the XLA dependency, then check parity with XLA. The first XLA run downloads GoMLX's XLA plugin, which takes about a minute:

```
go mod tidy
go run ./cmd/predict-digits -backend xla -runs 0
```

You should see `Backend:  xla` and `PASS: labels 10000/10000 identical`.

3. Run all the tests, now including the program's end-to-end tests, which use XLA by default on your Mac:

```
go test ./...
```

Six lines should say `ok` (`assets` shows "no test files", which is fine).

4. Commit the dependency update:

```
git add go.mod go.sum
git commit -m "T12: go.sum with the XLA backend; XLA parity checked on the Mac"
git push
```

5. T15: measure everything. Plug in the Mac and close other apps first. This takes about 5 minutes and writes everything to `results/`:

```
scripts/measure.sh
scripts/tables.sh > results/tables.md
cat results/tables.md
```

6. Commit the results:

```
git add results
git commit -m "T15: measurements on the Mac"
git push
```

Paste the output of steps 2, 3 and 5. Once the results are in, I'll write the README around them, and you'll build and open the executables (T17).

```
?   github.com/mioluigi/go-onnx-production/assets [no test files]
ok  github.com/mioluigi/go-onnx-production/cmd/measure 0.250s
ok  github.com/mioluigi/go-onnx-production/cmd/predict-digits 5.793s
ok  github.com/mioluigi/go-onnx-production/idx (cached)
ok  github.com/mioluigi/go-onnx-production/inference 0.313s
ok  github.com/mioluigi/go-onnx-production/parity (cached)
```

All six test packages pass on your Mac. One thing step 3 can't show: the no-argument test passes whether the program used XLA or fell back to pure Go. Step 2's output tells us which, so I'll check what has reached GitHub.

Ran a command ▸

Steps 1 to 3 are done: the leftover branch is gone and all tests pass on your Mac. Nothing from step 4 has reached GitHub yet.

Please paste the output of step 2 (`go run ./cmd/predict-digits -backend xla -runs 0`). It's the only proof that XLA itself works on your Mac, because the tests in step 3 pass even when the program quietly falls back to pure Go. Look for:

- `Backend:  xla`, not `go`
- `PASS: labels 10000/10000 identical`

If step 2 shows both, go ahead with steps 4, 5 and 6 (commit `go.sum`, run `scripts/measure.sh` and `scripts/tables.sh`, commit `results/`), and paste the `cat results/tables.md` output.

If step 2 failed or showed a warning about XLA, stop there and paste it. `scripts/measure.sh` needs XLA working.

```
[main 340a578] T15: measurements on the Mac
 5 files changed, 31 insertions(+)
 create mode 100644 results/go-bench-xla.txt
 create mode 100644 results/go-xla.txt
 create mode 100644 results/machine.txt
 create mode 100644 results/python.txt
 create mode 100644 results/tables.md
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 10 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 1.55 KiB | 1.55 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/mioluigi/go-onnx-production.git
   3b8988f..340a578  main -> main
```

Show more

Checking the file set for missing benchmark outputs.

The commit has only five result files. `go-go.txt`, the pure-Go benchmark, the profiles and `whole-program.txt` are missing, so `measure.sh` probably stopped partway. Let me look at what it saved.

Created a file, ran 4 commands ▸

All tests pass again; `inference` coverage is 92.1 %. I'll also make `measure.sh` say plainly when a step fails, because last time it stopped without telling you. Then record the decision and push.

Commit and push the benchmark fix ▸

The fix is on GitHub. Your XLA results so far already change the story, so here they are before you rerun.

On your M1 Max, from the measurements that did complete:

	Python, PyTorch	Python, ONNX Runtime	Go, GoMLX XLA
Load model	980 ms	59 ms	77 ms
First prediction	4.5 ms	0.8 ms	153 ms
10,000 images, one call	3.8 ms	5.2 ms	15.3 ms
One image, median	28.1 µs	11.8 µs	40.8 µs

All three agree on every digit; Go/XLA's largest probability difference from PyTorch is 7.75e-07. Go is clearly **slower** at the prediction work itself. Its advantages are elsewhere, such as loading time, and the whole-program numbers will show memory and start-up. The README and the recommendation to management will say that plainly.

Rerun T15. The `&&` means the tables are built only if every measurement succeeds:

```
cd ~/Projects/go-onnx-production
source .venv/bin/activate
git pull
scripts/measure.sh && scripts/tables.sh > results/tables.md
cat results/tables.md
```

If it ends with `STOPPED because the step above failed`, paste the output and don't commit. Otherwise, commit and push:

```
git add results
git commit -m "T15: measurements on the Mac, all backends"
git push
```

Then paste `results/tables.md` here, and I'll start the README (T19).

Pasted text.txt

TXT

Suspecting a hidden compile cache behind the timing gap ▸

All the measurements completed. Before writing the README from them I checked that the tables agree with each other, and one pair doesn't.

Check whether the T15 results were pushed ▸

Your full results aren't on GitHub yet; the commit-and-push step hasn't run. Hold off for one minute, because two numbers disagree and I want to know why before the README depends on them.

- **The timing run.** For Go with XLA it measured a 78 ms load plus a 177 ms first prediction. Its logs put about 350 ms between program start and the end of the parity check.
- **The whole-program table.** It says the same work (`predict-digits -backend xla -runs 0`) takes 158 ms in total, which is less than load plus first prediction alone.

The pure-Go numbers agree with each other (about 105 ms vs 112 ms), so the question is specific to XLA. One possibility is that XLA's first-call cost varies a lot between runs. I'd rather measure it than guess. This takes about 15 seconds:

```
cd ~/Projects/go-onnx-production
for i in 1 2 3 4 5; do /usr/bin/time -p dist/predict-digits -backend xla -runs 1
2>&1 | grep -E "Load model|First prediction|^real"; done
```

Each of the five runs prints three lines: the load time, the first-prediction time, and `real`, the whole run's wall time in seconds. Paste all of it.

Then commit and push the results as they are, since `results/` must hold what `measure.sh` produced:

```
git add results
git commit -m "T15: measurements on the Mac, all backends"
git push
```

Meanwhile I'll start the README (T19). Everything else in the tables is consistent, and three findings are already clear:

- **Parity holds everywhere.** All four runtimes give the same digit for all 10,000 images, within 1.5e-6 of PyTorch.
- **Python's libraries are faster at the prediction work itself.** ONNX Runtime has the lowest single-image latency (10 µs) and PyTorch the fastest bulk prediction (3.7 ms). GoMLX with XLA takes 15.3 ms for bulk, and pure Go 47 ms.
- **Go wins on start-up and memory.** A complete Go run with the pure-Go backend takes 112 ms and peaks at 154 MB. The same job in Python with PyTorch takes 1,474 ms and 253 MB.

```
[main efc1b97] T15: measurements on the Mac, all backends
 10 files changed, 203 insertions(+), 14 deletions(-)
```

```
 create mode 100644 results/cpu-profile.txt
 create mode 100644 results/go-bench-go.txt
 create mode 100644 results/go-go.txt
 create mode 100644 results/memory-profile.txt
 create mode 100644 results/whole-program.txt
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 10 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 5.80 KiB | 2.90 MiB/s, done.
Total 13 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/mioluigi/go-onnx-production.git
  7154347..efc1b97  main -> main
```

Show more

The full results are on GitHub. The diagnostic loop's output wasn't in your message, so I'll read the new result files for clues to the XLA discrepancy while I start the README.

Created 2 files, ran 6 commands ▸

The README is on GitHub. It has the recommendation, the generated results tables, concerns, instructions, testing, executables and the process record.

The recommendation, in short: **yes, develop in Python and serve in Go, with a parity check before every release, but for Go's operational benefits, not for speed.** Python's runtimes predict faster than GoMLX on this model. Go starts in about a tenth of a second against 1.5 s for PyTorch, uses less memory, and ships as one executable.

Five things are left before you submit. Run `git pull` first.

1. The XLA check (15 seconds). The whole-program time for Go with XLA (158 ms) is less than its own load plus first prediction. Until the diagnostic explains it, I can't stand behind that one number:

```
for i in 1 2 3 4 5; do /usr/bin/time -p dist/predict-digits -backend xla -runs 1
2>&1 | grep -E "Load model|First prediction|^real"; done
```

2. T17: build and open the executables.

```
scripts/build.sh
open dist/PredictDigits.app
```

A Terminal window should show PASS and the timings. The `.exe` can only be built here; it needs a Windows machine to run.

3. **T20: Gemini repository review.** In Gemini on the web, share the repo, either with its GitHub link if Gemini can read it or by uploading `README.md`, `cmd/predict-digits/main.go`, `inference/inference.go` and `results/tables.md`. Ask:

> *Review this repository against the attached rubric (docs/assignment-and-rubric.md). List concrete problems in code, tests, documentation and the recommendation, most important first. Check that README claims match results/tables.md.*

Then paste Gemini's answer here, and I'll triage it as before.

4. **T18: AI dialogs.** Save each conversation as a PDF (browser **Print → Save as PDF**) into `docs/ai-dialogs/`:

- this Claude conversation
- Gemini's spec review
- Gemini's repository review

5. **Service tiers.** The assignment asks for free AI tools, and the README's disclosure line is marked ⚠ **TO CONFIRM**. Tell me which plan you used for Claude and for Gemini, and I'll write it exactly as you state it.

> For T20 how do I enable gemini on the web to have access the the share repo or do I have all of the files on my local drive to share?

Searched the web, read a page ▸

That page doesn't say whether the feature is in the free tier. It also matters whether your repo is public, so I'll check.

Check the repository's visibility ▸

You don't need to upload files from your Mac. Your repository is **public**, which is the case Gemini handles most simply, and at about 2.5 MB it's far below Gemini's limit of 100 MB and 5,000 files. Gemini imports it directly from GitHub:

1. Go to **gemini.google.com** on your Mac (computer only, not the phone app).
2. In the prompt box, click **Add file** (the +), then **More Uploads**, then **Import code**.
3. Paste `https://github.com/mioluigi/go-onnx-production` and click **Import**.
4. Type the review prompt from my last message and submit.

Two caveats:

- **Gemini takes a snapshot.** It won't see anything pushed after the import. The README is already on GitHub, so importing now is fine; if we change things later, a fresh chat would need a new import.

- **The free tier is uncertain.** Google's help page doesn't say whether Import code is in the free plan. It also requires **Gemini Apps Activity** (shown as "Keep Activity") to be turned on. If you don't see **Import code** under **More Uploads**, that's probably why.

Fallback if Import code isn't offered: upload a few files from your Mac instead. You already have them all in `~/Projects/go-onnx-production`. Run `git pull` first, then click **Add file** → **Upload files** and choose these four:

- `README.md`
- `docs/assignment-and-rubric.md`
- `cmd/predict-digits/main.go`
- `inference/inference.go`

Also attach `results/tables.md`, so Gemini can check the README's figures against the measurements.

Either way, paste Gemini's answer here and I'll triage it.

Sources:

- Gemini Apps Help: attach a GitHub repository to your prompt
- Google Workspace Updates: upload code folders and GitHub repositories to Gemini (June 2025)



Open questions on LLM free - Document that by default I already have Pro subscription for Claude and Ultra Subscription for Gemini. Here is the rigorous audit from Gemini attached.

Created a file, ran 6 commands ▸

Gemini's finding 6: add a concurrency benchmark (many simultaneous single-image requests) before submitting?

Document only

Created a file, ran 7 commands ▸

T20 is done and on GitHub. Gemini's review was triaged and recorded, and your tier statement is in the README. Four steps remain before you submit. Run `git pull` first.

1. The XLA check (15 seconds). Until this runs, I can't stand behind the whole-program time for Go with XLA (158 ms), which is less than its own load plus first prediction:

```
for i in 1 2 3 4 5; do /usr/bin/time -p dist/predict-digits -backend xla -runs 1
2>&1 | grep -E "Load model|First prediction|^real"; done
```

2. T17: build and open the executables.

```
scripts/build.sh
open dist/PredictDigits.app
```

A Terminal window should show PASS and the timings. Tell me if it doesn't.

3. T18: AI dialogs. Gemini's repository review is already in `docs/ai-dialogs/`. Save two more as PDFs (browser **Print → Save as PDF**) into that folder:

- `claude-build-dialog.pdf`: this conversation
- `gemini-spec-review.pdf`: Gemini's spec review from earlier today

Then push them:

```
git add docs/ai-dialogs
git commit -m "T18: AI dialogs"
git push
```

4. T21: submit. Once those are pushed, I'll run a final check on a fresh clone. Then paste `https://github.com/mioluigi/go-onnx-production` into the Canvas comments.

Paste the output of steps 1 and 2 when you have them.

> Need more clarity with T18 - step 3 here. I have labeled the two gemini review: 1. gemini-spec-review.pdf and 2. gemini-fina-review.pdf. Guide me to complete this step

Sorting out which Gemini review file belongs where.

Gemini's final review is already in the repository: it's the PDF you sent me, saved as `docs/ai-dialogs/gemini-t20-repository-review.pdf`. So from your two files you only need to add `gemini-spec-review.pdf`.

For the Claude dialog, printing a long conversation full of tool runs from the browser often produces an incomplete PDF. The full record of this session is in my workspace, so I can write the transcript to a text file myself. Let me check it.

Ran 2 commands ▸

That transcript only covers the last few hours. Everything before today's 2:30 pm (the spec, plan and tasks T01 to T08) isn't in my workspace any more. **The complete conversation is in your Claude app**, so the Claude PDF has to come from there. Here's the whole step:

A. Save this Claude conversation as a PDF

1. Open this conversation in the Claude app or at claude.ai in your browser.
2. Scroll all the way to the top, so the whole conversation loads. It starts with your Week 5 assignment PDF.
3. Press **Cmd+P**, choose **Save as PDF**, name it `claude-build-dialog.pdf`, and save it to Downloads.
4. Open the PDF and check that it runs from the assignment message to the end. If part is missing, tell me and I'll add a text transcript for the part I still have.

B. Decide about `gemini-fina-review.pdf`

- The copy already in the repository shows only Gemini's answer, not the prompt you typed.
- If your file also shows your prompt, it's the better record of the dialog: use it to replace the existing copy (block C, last line).
- If it shows only Gemini's answer, skip it; it's a duplicate.

C. Move the files into the repository. This assumes they're in Downloads. If not, type `mv` and drag the file from Finder into Terminal to fill in its path.

```
cd ~/Projects/go-onnx-production
git pull
mv ~/Downloads/gemini-spec-review.pdf docs/ai-dialogs/
mv ~/Downloads/claude-build-dialog.pdf docs/ai-dialogs/
```

Only if B said to replace:

```
mv ~/Downloads/gemini-fina-review.pdf docs/ai-dialogs/gemini-t20-repository-review.pdf
```

D. Check, commit and push:

```
ls docs/ai-dialogs
git add docs/ai-dialogs
git commit -m "T18: AI dialogs (Claude build conversation, Gemini spec review)"
git push
```

`ls` should list three PDFs: `claude-build-dialog.pdf`, `gemini-spec-review.pdf` and `gemini-t20-repository-review.pdf`.

After your push I'll add a short README in that folder describing each file, then run the final fresh-clone check (T21).

Two earlier steps are still open:

- the five-run XLA check
- `scripts/build.sh` plus opening the `.app` (T17)

Paste their output when you can.